

Reviewer Pack — MST Persistence Entropy Lower-Bounds Log-Rank of Sign Matric...

Entry 007b80963b09 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 11:03:43 UTC

MST Persistence Entropy Lower-Bounds Log-Rank of Sign Matrices

Entry ID: 007b80963b09 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 11:03:43 UTC

1. Conjecture

Field A (mathematical branch): Topological data analysis — persistent H_0 entropy of single-linkage hierarchical clustering on a finite metric space (Carlsson barcode theory; Edelsbrunner-Harer; Atienza-Gonzalez-Diaz-Soriano-Trigueros persistence-entropy) **Field B** (complexity object): Rank / log-rank lower bounds for ± 1 sign matrices M_f arising as communication matrices in deterministic and randomized two-party communication complexity (Razborov, Sherstov, Klartag rigidity tradition)

Statement:

For every ± 1 matrix M of size $n \times n$ with $4 \leq n \leq 40$, let T be the minimum spanning tree of the rows of M under un-normalized Hamming distance, let $\ell_1, \dots, \ell_{n-1}$ be its edge weights, restrict to the multiset $P = \{\ell_i : \ell_i > 0\}$, set $L = \sum_{\ell \in P} \ell$, and define the persistence entropy $PE(M) = -\sum_{\ell \in P} (\ell/L) \cdot \log_2(\ell/L)$ (with $PE(M) = 0$ if P is empty). Then $PE(M) \leq \log_2(\text{rank}_{\mathbb{R}}(M) + 1)$; equivalently $2^{\{PE(M)\}} \leq \text{rank}_{\mathbb{R}}(M) + 1$. A single ± 1 matrix witnessing $2^{\{PE(M)\}} > \text{rank}_{\mathbb{R}}(M) + 1$ refutes the conjecture.

Rationale (proposer's reasoning):

Persistent H_0 barcodes / MST edge-length spectra are a multiscale fingerprint of how rows of M cluster in Hamming space; n vectors that span an r -dimensional $\mathbb{R}$ -subspace can only realise an MST whose edge lengths concentrate on $O(r)$ effective scales, so their Shannon entropy ought to be log-bounded by the rank. Because PE is a metric/topological functional of the row cloud and not a polynomial in the entries, the bridge sidesteps algebraization, and because we lower-bound matrix rank (a communication-specific quantity, not poly-size circuits) it sidesteps Razborov-Rudich naturalness while still feeding directly into log-rank / discrepancy lower bounds. To our knowledge persistent-homology entropy has not been previously deployed against communication-matrix rank, putting the field_A $\times$ field_B pair below the <5-paper threshold.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 995069bec73cbb55

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 30 seeds $\times n \in \{8, 16, 24, 32\} \times 5$ families (~ 600 matrices), every trial must satisfy $2^{\{\text{PE}(M)\}} \leq \text{rank}_{\mathbb{R}}(M) + 1$. Report violation_fraction and worst_slack = min over trials of $(\text{rank}_{\mathbb{R}}(M) + 1 - 2^{\{\text{PE}(M)\}})$. Supported iff zero violations; a single negative slack falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - persistence entropy minimum spanning tree Hamming sign matrix rank-persistent homology barcode communication complexity log-rank lower bound-single-linkage hierarchical clustering entropy rank Boolean matrix rigidity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    rank = 0
    for i in range(n):
        if rank < n and A[i][i] == 0:
            pivot_found = False
            for k in range(i, n):
                if A[k][i] != 0:
                    A[i], A[k] = A[k], A[i]
                    pivot_found = True
                    break
            if not pivot_found:
                continue
        if A[i][i] == 0:
            continue
        rank += 1
```

```

        for k in range(n):
            if k != i and A[k][i] != 0:
                factor = -A[k][i] / A[i][i]
                for j in range(i, n):
                    A[k][j] += factor * A[i][j]
    return rank

def hamming_distance(a, b):
    return sum(x != y for x, y in zip(a, b))

def persistence_entropy(M):
    n = len(M)
    distances = []
    for i in range(n):
        for j in range(i + 1, n):
            distances.append(hamming_distance(M[i], M[j]))
    if not distances:
        return 0
    L = sum(distances)
    P = [d / L for d in distances]
    PE = -sum(p * math.log2(p) for p in P)
    return PE

def rank_real(M):
    M_copy = [row[:] for row in M]
    rank = gaussian_elimination(M_copy)
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([8, 16, 24, 32])
    family = random.randint(0, 4)

    if family == 0: # i.i.d. Bernoulli ±1
        M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
    elif family == 1: # sign(AB^T) with A,B Gaussian of inner rank r ∈ {1,2,4,8}
        r = random.choice([1, 2, 4, 8])
        A = [[random.gauss(0, 1) for _ in range(r)] for _ in range(n)]
        B = [[random.gauss(0, 1) for _ in range(n)] for _ in range(r)]
        M = [[sum(a * b for a, b in zip(A[i], B[j])) for j in range(n)] for i in range(n)]
    elif family == 2: # Sylvester-Hadamard for n ∈ {8,16,32}
        if n not in [8, 16, 32]:
            return {"metric_name": "PE", "metric_value": -math.inf, "instances_tested": 0, "conjecture": "rank < 2"}
        M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
    elif family == 3: # random ±1 circulants
        M = [[M[i - j % n][j] for j in range(n)] for i in range(n)]
    else: # row-repeated Hadamard blocks (forced low-rank, wide distance spread – the natural advantage of low-rank)
        block_size = n // 4
        M = []
        for _ in range(4):
            block = [[random.choice([-1, 1]) for _ in range(block_size)] for _ in range(block_size)]
            M.extend(block)

    PE = persistence_entropy(M)
    rank = rank_real(M)
    if 2 ** PE > rank + 1:

```

```

        return {"metric_name": "PE", "metric_value": PE, "instances_tested": 1, "conjecture_holds": True}
    else:
        return {"metric_name": "PE", "metric_value": PE, "instances_tested": 1, "conjecture_holds": False}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_PE = sum(result["metric_value"] * result["instances_tested"] for result in results) / sum(result["instances_tested"] for result in results)
    std_PE = math.sqrt(sum((result["metric_value"] - mean_PE)**2 * result["instances_tested"] for result in results) / sum(result["instances_tested"] for result in results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_PE} std={std_PE} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        worst_slack = min(result["metric_value"] * result["instances_tested"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={first_failing_seed}^2PE > rank+1 first_failing_slack={worst_slack}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1955174d.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1955174d.py", line 87, in run_trial
    PE = persistence_entropy(M)
        ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1955174d.py", line 54, in persistence_entropy
    PE = -sum(p * math.log2(p) for p in P)
          ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1955174d.py", line 54, in <genexpr>
    PE = -sum(p * math.log2(p) for p in P)
              ~~~~~^~~~~
ValueError: math domain error

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError (math domain error) in persistence_entropy due to a log2(p) call with p=0, before producing any RESULT line or trial data. No violation_fraction or worst_slack was computed, so the pre-registered support condition cannot be evaluated. | next: Fix persistence_entropy to filter $p > 0$ strictly (or skip zero-probability terms) and re-run the full 30 seeds $\times$ 4 sizes $\times$ 5 families grid to obtain violation_fraction and worst_slack.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	341605
2	preregistration	claude_max	opus	0	0	5697
3	novelty	claude_max	opus	0	0	3463
4	novelty	claude_max	opus	0	0	5883
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17310
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12344
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17801
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15117
9	judge	claude_max	opus	0	0	5051

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 424270 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/007b80963b09.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/007b80963b09.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/007b80963b09.tar.gz` (if generated)